

SecurAx Scanner Engine

Base Documentaire	Aligné sur SCANNERS_TECHNICAL.md (Architecture sans contraintes d'hébergement).
Philosophie	Autocritique constante : repousser les limites techniques pour l'excellence.
Langue	Français
Date	Juin 2026

1. SCANNER WEB APPLICATION (web_scanner.py)

1.1 Capacités et Périmètre d'Action

Audit passif rapide de la couche HTTP basé sur une requête initiale (GET).

Fonctionnalité	Détail de l'analyse
Headers sécurité	Vérifie HSTS, CSP, X-Frame-Options, X-Content-Type-Options, Referrer-Policy.
Empreinte serveur	Analyse les headers Server et X-Powered-By.
Redirection HTTPS	Vérifie si HTTP redirige vers HTTPS de façon sécurisée (301/302).
Cookies	Contrôle les attributs Secure, HttpOnly et SameSite.

1.2 Limites Réelles et Autocritique

Dans un souci d'amélioration continue, voici les limites actuelles du moteur :

Limite (Ce qu'il ne fait pas)	Explication technique
Pas de crawling	Ne vérifie que la racine du site, ignorant 99% du contenu.
Aucun test d'injection	Zéro payload envoyé, ne détecte pas de failles XSS ou SQLi réelles.
Pas de CORS check	Ne teste pas les politiques d'origines permissives (Access-Control-Allow-Origin: *).

1.3 Améliorations Proposées pour l'Architecture Cible

Pour dépasser ces limites (en considérant une infrastructure cloud dédiée et évolutive), voici les prochaines étapes de R&D; :

Axe d'Amélioration	Description Technologique / Impact
Crawler Léger Intégré	Intégrer BeautifulSoup pour extraire 10-20 sous-pages et vérifier la cohérence des headers sur tout le site (Priorité Haute).
Test CORS Actif	Envoyer des requêtes avec 'Origin: https://evil.com' pour capturer et alerter sur les configurations CORS permissives (Priorité Haute).

Axe d'Amélioration	Description Technologique / Impact
Audit Robots & Sitemap	Analyser robots.txt et sitemap.xml pour révéler la structure interne et les chemins sensibles exposés (Priorité Moyenne).

2. SCANNER DAST MULTI-MOTEURS (dast_scanner.py)

2.1 Capacités et Périmètre d'Action

Orchestrateur d'analyse dynamique utilisant 3 moteurs externes (OWASP ZAP, Nuclei, Nikto) et un moteur natif de secours.

Fonctionnalité	Détail de l'analyse
OWASP ZAP	Démon + API REST — Spidering actif + injections XSS/SQLi/CSRF.
Nuclei	CLI/Cloud — Plus de 9000 templates YAML orientés CVE et expositions.
Nikto	Vérification des configurations des serveurs web et des fichiers obsolètes.
Fallback Natif	Utilise `requests` avec des payloads basiques si les moteurs externes sont inactifs.

2.2 Limites Réelles et Autocritique

Dans un souci d'amélioration continue, voici les limites actuelles du moteur :

Limite (Ce qu'il ne fait pas)	Explication technique
Temps d'exécution massif	Les moteurs ZAP + Nuclei peuvent prendre 15 à 30 minutes sur un site moyen.
Authentification Complexe	Incapable de scanner derrière des portails SSO ou MFA sans Selenium.
Blocage WAF (Faux Négatif)	Le trafic massif est souvent bloqué comme une attaque DDoS, aveuglant le scanner.
Approche Boîte Noire	Déduit les failles uniquement via le comportement HTTP, sans voir le code source.

2.3 Améliorations Proposées pour l'Architecture Cible

Pour dépasser ces limites (en considérant une infrastructure cloud dédiée et évolutive), voici les prochaines étapes de R&D; :

Axe d'Amélioration	Description Technologique / Impact
Support Auth Bearer/JWT	Accepter des tokens de session et les injecter dynamiquement dans les requêtes pour scanner les zones authentifiées (Priorité Haute).
Throttle Intelligent	Ralentir dynamiquement le scan si le serveur répond 429 (Too Many Requests) pour éviter le bannissement.
IAST Integration (Agent-based)	Proposer une option 'Agent IAST' que le client peut installer sur son serveur cible pour lier l'attaque DAST à la ligne de code source exacte.

3. SCANNER SAST MULTI-LANGAGES (sast_scanner.py)

3.1 Capacités et Périmètre d'Action

Analyse statique concurrente de code source via 4 moteurs dédiés (Bandit, Semgrep, Gitleaks, Natif).

Fonctionnalité	Détail de l'analyse
Bandit (Python)	Linteur sécurité dédié détectant eval(), exec(), subprocess et injections SQL.
Semgrep (Polyglotte)	Analyse AST multi-langages avec les règles OWASP Top 10 et CWE Top 25.
Gitleaks	Détection de secrets hardcodés (clés AWS, tokens GitHub, Stripe) sur tous les fichiers.
Scanner Natif	Fallback regex ciblant les patterns critiques même sans dépendances système.

3.2 Limites Réelles et Autocritique

Dans un souci d'amélioration continue, voici les limites actuelles du moteur :

Limite (Ce qu'il ne fait pas)	Explication technique
Faux positifs structurels	Alarme sur du code mort (dans /tests/) ou des variables mockées ('test_password').
Manque de Taint Tracking	Analyse par fichier : ne suit pas le flux de données entre plusieurs modules ou microservices.
Contexte d'exécution ignoré	Signale des failles qui pourraient être mitigées par des WAF ou des wrappers globaux.

3.3 Améliorations Proposées pour l'Architecture Cible

Pour dépasser ces limites (en considérant une infrastructure cloud dédiée et évolutive), voici les prochaines étapes de R&D :

Axe d'Amélioration	Description Technologique / Impact
Filtres IA (ARIA)	Envoyer les résultats bruts au moteur IA pour évaluer la pertinence réelle et éliminer les faux positifs avant de les présenter à l'utilisateur (Déjà disponible).

Axe d'Amélioration	Description Technologique / Impact
Taint Analysis (CodeQL)	Intégrer CodeQL pour réaliser un suivi complet du flux d'exécution entre l'entrée utilisateur et l'exécution en base de données.
Scan Diff & .gitignore	Ignorer automatiquement node_modules/ et ne scanner que les fichiers modifiés via `git diff` pour une intégration CI/CD ultrarapide.

4. SCANNER RÉSEAU & PORTS (netscan_scanner.py)

4.1 Capacités et Périmètre d'Action

Cartographie de la surface d'attaque réseau externe via Nmap (wrapper python-nmap).

Fonctionnalité	Détail de l'analyse
Scan de Ports TCP	Découverte des services exposés avec un scan furtif SYN (Half-open).
Fingerprinting	Détection précise des OS et des bannières logicielles (-sV -O).
Scripts NSE	Exécution de scripts Nmap pour valider des vulnérabilités connues (CVE).

4.2 Limites Réelles et Autocritique

Dans un souci d'amélioration continue, voici les limites actuelles du moteur :

Limite (Ce qu'il ne fait pas)	Explication technique
Bloqué par IDS/IPS	Les scans massifs déclenchent les pare-feu et les systèmes de prévention d'intrusion.
Pas de contexte Applicatif	Sait que le port 80 est ouvert, mais n'inspecte pas l'arborescence web.
Requis Système	Nécessite Nmap et potentiellement des raw sockets (root) pour un scan optimal.

4.3 Améliorations Proposées pour l'Architecture Cible

Pour dépasser ces limites (en considérant une infrastructure cloud dédiée et évolutive), voici les prochaines étapes de R&D; :

Axe d'Amélioration	Description Technologique / Impact
IP Rotation & Tor	Distribuer le scan sur des proxies Tor/VPN pour échapper au blocage IP (À condition d'avoir l'autorisation explicite du client).
Timing Évasif	Intégrer `--timing polite` (-T2) pour espacer les requêtes et passer sous le radar des IDS.

Axe d'Amélioration	Description Technologique / Impact
Scan Natif Multi-thread	Implémenter <code>`socket.connect_ex()`</code> en parallèle pur Python comme fallback si les privilèges root/nmap manquent.

5. SCANNER SSL / TLS (ssl_scanner.py)

5.1 Capacités et Périmètre d'Action

Audit cryptographique de la couche de transport basé sur la librairie `ssl` native.

Fonctionnalité	Détail de l'analyse
Expiration	Alerte si le certificat expire dans moins de 30 jours.
Protocoles	Détecte l'utilisation de SSLv2/v3 et TLS 1.0/1.1 (dépréciés).
Ciphers faibles	Vérifie l'absence de Forward Secrecy et les algorithmes vulnérables.
Parsing ASN.1	Extrait SAN, issuer, et dates directement du certificat X.509.

5.2 Limites Réelles et Autocritique

Dans un souci d'amélioration continue, voici les limites actuelles du moteur :

Limite (Ce qu'il ne fait pas)	Explication technique
Ciblé sur le port 443	Ignore les autres ports qui utilisent potentiellement TLS (ex: 8443, 465, 5432).
Pas de test POODLE/Heartbleed	Ces failles requièrent des handshakes volontairement malformés que la librairie `ssl` native ne permet pas.
Pas de vérification CT	Ne vérifie pas si le certificat est présent dans les logs de Certificate Transparency.

5.3 Améliorations Proposées pour l'Architecture Cible

Pour dépasser ces limites (en considérant une infrastructure cloud dédiée et évolutive), voici les prochaines étapes de R&D; :

Axe d'Amélioration	Description Technologique / Impact
Intégration SSLyze	Remplacer l'approche native par la bibliothèque pure Python `SSLyze` pour détecter Heartbleed, ROBOT, BEAST, POODLE (Priorité Haute).
CT Log Check API	Interroger `crt.sh` via API pour découvrir des sous-domaines cachés liés au certificat principal.

Axe d'Amélioration	Description Technologique / Impact
Multi-Ports	Coupler avec le Network Scanner pour vérifier TLS sur l'ensemble des ports ouverts.

6. SCANNER DE DÉPENDANCES (dep_scanner.py)

6.1 Capacités et Périmètre d'Action

Supply Chain Analysis via requêtes à l'API Google OSV.dev.

Fonctionnalité	Détail de l'analyse
SCA & SBOM	Parse requirements.txt, package.json, composer.json et extrait les versions exactes.
Mapping CVE	Croise l'inventaire avec la base de données OSV.dev en temps réel.
Détection de Malwares	Repère les dépendances obsolètes, abandonnées ou potentiellement typosquatées.

6.2 Limites Réelles et Autocritique

Dans un souci d'amélioration continue, voici les limites actuelles du moteur :

Limite (Ce qu'il ne fait pas)	Explication technique
Faux positifs structurels	Signale la librairie entière vulnérable, même si la fonction spécifique affectée n'est pas appelée.
Dépendances Transitives	Sans fichier lock strict, les sous-dépendances critiques sont ignorées par le parser Regex basique.
Rate Limiting API	Des centaines de requêtes individuelles peuvent saturer l'API OSV.

6.3 Améliorations Proposées pour l'Architecture Cible

Pour dépasser ces limites (en considérant une infrastructure cloud dédiée et évolutive), voici les prochaines étapes de R&D; :

Axe d'Amélioration	Description Technologique / Impact
Batch OSV API	Envoyer les packages par lots de 100 via une seule requête POST pour optimiser le temps d'exécution (Priorité Haute).
Génération SBOM (Syft)	Utiliser `Syft` en backend pour résoudre dynamiquement l'arbre de dépendances complet même sans fichier lock.

Axe d'Amélioration	Description Technologique / Impact
Reachability Analysis	Croiser l'arbre des dépendances avec l'AST généré par Bandit/Semgrep pour confirmer que la fonction vulnérable est effectivement appelée.

7. AUDIT DE CONFIGURATION INTERNE (server_int.py)

7.1 Capacités et Périmètre d'Action

Analyse des fichiers locaux de configuration (Nginx/Apache/SSH) via Regex.

Fonctionnalité	Détail de l'analyse
Masquage (Nginx)	Vérifie la présence de <code>`server_tokens off`</code> .
Durcissement SSH	Vérifie que <code>`PermitRootLogin no`</code> est configuré dans <code>sshd_config</code> .
TLS Serveur	Vérifie <code>`ssl_protocols TLSv1.2+`</code> pour forcer des protocoles robustes.

7.2 Limites Réelles et Autocritique

Dans un souci d'amélioration continue, voici les limites actuelles du moteur :

Limite (Ce qu'il ne fait pas)	Explication technique
Analyse Statique Exclusive	Lit le disque mais ne vérifie pas la configuration réellement chargée en RAM par les démons.
Inclusions Externes	Ne suit pas les répertoires inclus via <code>`include conf.d/*`</code> ou <code>`IncludeOptional`</code> .
Limitation des Regex	Les expressions régulières sont fragiles face à une syntaxe Nginx complexe ou multi-lignes.

7.3 Améliorations Proposées pour l'Architecture Cible

Pour dépasser ces limites (en considérant une infrastructure cloud dédiée et évolutive), voici les prochaines étapes de R&D; :

Axe d'Amélioration	Description Technologique / Impact
Parseur Nginx Complet	Intégrer un parseur comme <code>`crossplane`</code> qui construit l'arbre de configuration complet et fusionne toutes les inclusions externes (Priorité Haute).
Apache ConfigParser	Utiliser un parseur structuré au lieu de Regex pour lire la hiérarchie Apache HTTPD.

Axe d'Amélioration	Description Technologique / Impact
Vérification Runtime	Si applicable, se connecter à l'API Docker locale pour auditer les paramètres de runtime des conteneurs hébergeant ces serveurs.

8. SONDAGE SERVEUR EXTERNE BLACK-BOX (server_ext.py)

8.1 Capacités et Périmètre d'Action

Audit externe complet sans accès SSH — connexions HTTP parallèles.

Fonctionnalité	Détail de l'analyse
Détection de version	Récupère Apache/Nginx depuis le header Server et effectue un mapping CVE de base.
Méthodes HTTP	Teste l'activation des méthodes dangereuses (TRACE, PUT, CONNECT).
Chemins sensibles	Détecte l'exposition de /admin, /.git/, /phpinfo.php, /.env via ThreadPoolExecutor.
GeoIP & Hébergeur	Identifie l'ASN et le provider cloud de l'IP cible.

8.2 Limites Réelles et Autocritique

Dans un souci d'amélioration continue, voici les limites actuelles du moteur :

Limite (Ce qu'il ne fait pas)	Explication technique
Header Obfuscation	Une configuration propre `server_tokens off` masque la version réelle, rendant le scanner aveugle.
WAF Honeypot	Les Web Application Firewalls renvoient souvent des 200 OK factices pour les chemins sensibles, perturbant la détection.
Surface Limitée	Se base sur un dictionnaire codé en dur ; il ne découvrira pas des chemins exotiques propres à l'entreprise.

8.3 Améliorations Proposées pour l'Architecture Cible

Pour dépasser ces limites (en considérant une infrastructure cloud dédiée et évolutive), voici les prochaines étapes de R&D :

Axe d'Amélioration	Description Technologique / Impact
Détection WAF Avancée	Utiliser des heuristiques HTTP (fingerprinting des réponses Cloudflare/Akamai) pour déterminer si on audite le serveur ou son bouclier (Priorité Haute).

Axe d'Amélioration	Description Technologique / Impact
Subdomain Takeover	Vérifier si les domaines pointent vers des services CNAME orphelins (ex: buckets S3 supprimés).
Fuzzing intelligent	Étendre le dictionnaire de chemins sensibles dynamiquement en fonction du type de serveur détecté.

9. SCANNER DOCKER SECURITY (docker_scanner.py)

9.1 Capacités et Périmètre d'Action

Analyse statique pure Python des `Dockerfile` et `docker-compose.yml`.

Fonctionnalité	Détail de l'analyse
Privilèges	Repère `USER root`, `privileged: true`, et les montages de `/var/run/docker.sock`.
Supply Chain	Détecte les images `:latest` non épinglées et les patterns `curl   bash`.
Exposition	Identifie les ports sensibles exposés (SSH, MySQL, Docker API) et l'utilisation de `network_mode: host`.
Secrets	Trouve les clés/mots de passe dans les directives `ENV`.

9.2 Limites Réelles et Autocritique

Dans un souci d'amélioration continue, voici les limites actuelles du moteur :

Limite (Ce qu'il ne fait pas)	Explication technique
Analyse Statique Exclusive	Ne lance pas le conteneur ; ne détecte pas les vulnérabilités de runtime.
Pas de scan de l'Image OS	N'analyse pas les paquets (apt/apk) installés à l'intérieur de la couche système de l'image de base ubuntu/alpine.
Faux positifs (ENV)	Peut signaler des variables de templates (\$SECRET) comme étant des valeurs codées en dur.

9.3 Améliorations Proposées pour l'Architecture Cible

Pour dépasser ces limites (en considérant une infrastructure cloud dédiée et évolutive), voici les prochaines étapes de R&D :

Axe d'Amélioration	Description Technologique / Impact
Scanner d'Image (Grype/Trivy)	Au-delà du Dockerfile, télécharger l'image et analyser sa nomenclature (SBOM) pour détecter les failles logicielles internes.

Axe d'Amélioration	Description Technologique / Impact
Secrets dans COPY	Ajouter une logique pour détecter si des fichiers sensibles (.env, id_rsa) sont copiés (`COPY`) dans le build context.
Règles Hadolint	Implémenter formellement les règles de best practices Dockerfile (Hadolint) en Python.

10. SCANNER DNS & EMAIL SECURITY (dns_scanner.py)

10.1 Capacités et Périmètre d'Action

Audit de l'infrastructure de domaine et lutte anti-spoofing via DNS-over-HTTPS.

Fonctionnalité	Détail de l'analyse
Anti-Spoofing	Vérifie les protocoles SPF (-all vs +all) et DMARC (quarantine/reject/none).
DKIM & MX	Probing de 7 sélecteurs communs (google, mail, default) et validation des serveurs d'échange.
Sécurité DNS	Vérifie les enregistrements CAA (CAs autorisées), DS (DNSSEC) et teste le Zone Transfer (AXFR).

10.2 Limites Réelles et Autocritique

Dans un souci d'amélioration continue, voici les limites actuelles du moteur :

Limite (Ce qu'il ne fait pas)	Explication technique
Sélecteurs DKIM Inconnus	Ne trouve que les clés DKIM de la liste prédéfinie ; un sélecteur custom sera ignoré.
Limites de résolution SPF	Ne valide pas récursivement la limite stricte de 10 résolutions DNS (RFC 7208).
Validation DNSSEC	Vérifie l'existence des enregistrements mais ne valide pas mathématiquement la chaîne cryptographique.

10.3 Améliorations Proposées pour l'Architecture Cible

Pour dépasser ces limites (en considérant une infrastructure cloud dédiée et évolutive), voici les prochaines étapes de R&D; :

Axe d'Amélioration	Description Technologique / Impact
Sélecteurs DKIM Dynamiques	Extraire les sélecteurs DKIM à partir d'en-têtes d'emails capturés ou via le crawl du site web cible (Priorité Moyenne).

Axe d'Amélioration	Description Technologique / Impact
MX Security Score	Vérifier si les serveurs MX cibles sont listés dans des bases de données DNSBL (Listes noires de spam).
Support BIMl	Vérifier l'enregistrement DNS TXT BIMl (Brand Indicators for Message Identification), nouveau standard email d'authentification de marque.

11. SCANNER WORDPRESS

(wordpress_scanner.py)

11.1 Capacités et Périmètre d'Action

Audit passif et proactif ciblant la surface d'attaque spécifique aux CMS WordPress.

Fonctionnalité	Détail de l'analyse
Détection de Version	Identifie la version via readme.html ou la balise meta generator, et alerte sur les versions EOL (End-of-Life).
User Enumération	Interroge /wp-json/wp/v2/users pour lister les comptes, ciblant particulièrement l'existence de l'utilisateur 'admin'.
Vecteurs d'attaque	Vérifie si xmlrpc.php (Brute-force/SSRF) et wp-cron.php (DoS) sont exposés publiquement.
Fichiers Critiques	Sonde la présence publique de wp-config.php et /wp-content/debug.log.

11.2 Limites Réelles et Autocritique

Dans un souci d'amélioration continue, voici les limites actuelles du moteur :

Limite (Ce qu'il ne fait pas)	Explication technique
Pas de scan de plugins/thèmes	Ne liste ni n'audite les dizaines de plugins potentiellement installés dans /wp-content/plugins/.
Blocage WAF / Rate Limiting	Les pare-feux spécifiques (Wordfence, iThemes Security) bloquent rapidement les requêtes répétées sur l'API REST.
Aucune Authentification	Ne peut pas auditer les surfaces d'attaque post-authentification.

11.3 Améliorations Proposées pour l'Architecture Cible

Pour dépasser ces limites (en considérant une infrastructure cloud dédiée et évolutive), voici les prochaines étapes de R&D :

Axe d'Amélioration	Description Technologique / Impact
WPScan API Intégration	S'interfacer avec l'API publique WPScan pour mapper les plugins détectés avec des CVEs spécifiques (Priorité Moyenne).
Plugin Enumération	Créer un dictionnaire pour sonder passivement l'existence des 100 plugins les plus populaires (HTTP 200/403 sur /wp-content/plugins/X/).
Credential Stuffing Check	Tester basiquement si la mire de connexion répond à l'utilisateur 'admin/admin', sans pour autant faire de brute-force lourd.